

RADIX

Talent Match Hackathon

5 people · 5 roles · 1 integrated tool

Contents

What You're Building Today3

Why This Matters3

The Five Roles.....4

The Shared Data Contract (Starting Point).....6

Your Test Data.....7

The Day: Four Phases7

Tech Stack & Guidance8

What “Done” Looks Like.....8

Presentation.....8

What You're Building Today

RADIX Talent Match: a small, real, end-to-end tool that helps a candidate understand how ready they actually are for a specific job — built on the same 12-skillset framework and real hiring data RADIX already researched.

After you are done, someone should be able to sit down at your tool and:

- Upload a JD and see what skills it actually requires
- Upload a resume and see what skills it shows
- Build a full profile that goes beyond the resume — hackathons, certifications, preferred roles
- Hit “Talent Check” and see how they stack up against a specific company's bar, across all 12 skillsets
- Hit “Skill Match” and see exactly how well they match one specific job posting, with a score and a gap list

Five of you are each building one of those five pieces, alone, for two hours. Then you integrate. That's the whole day.

Why This Matters

Levels 1–2 built a 163-parameter research framework, a 12-skillset competency dictionary, and real hiring-round data for real companies. Level 3 taught you to validate data automatically. Levels 4–5 taught you to generate and orchestrate it with agents. All of that has been building the company's side of a placement platform.

Today flips the lens: same underlying skill-set data, same quality mindset, but built for the candidate trying to figure out if they're actually ready.

The Five Roles

Each role below is one person's job for Phase 2. Read all five, not just your own — you need to know what you're handing off to, and what's being handed to you.

1 · JD Analytics

Mission	Given a JD (PDF or Word), extract a structured list of the skills and technologies it requires.
Input	One of the 6 sample JDs in the given data, or any JD, as a PDF or DOCX file.
Output	A structured skill list in the shared format — see The Shared Data Contract.
Approach	A strong LangChain/LLM use case. Feed the JD text to an LLM with a prompt that maps requirements onto the 12 RADIX skill categories (Coding, DSA, OOD, Aptitude, Communication, AI, Cloud, SQL, SWE, System Design, Networking, OS) plus any named technologies. Plain keyword-matching alone will miss a lot — the JDs are written in natural prose on purpose.
Tip	The skill signal is concentrated in “Key Responsibilities” and “What We're Looking For” — start there.

2 · Resume Parsing

Mission	Given a resume (PDF or Word), extract key technologies, skills, and technical signal.
Input	One of the 4 sample resumes in the given data, or any resume, as a PDF or DOCX file.
Output	A structured skill list in the shared format, plus whatever structured fields you can reliably pull (education, projects, experience).
Approach	The same LLM-extraction pattern as JD Analytics, applied to a differently-shaped document. Talk to whoever's building #1 — you may be able to share real code, not just a data format.
Tip	Resumes are messier and less standardized than JDs. Build in tolerance for format variation rather than assuming one layout.

3 · Profile Builder

Mission	Let a candidate build a full profile: basic details, skills, hackathons/internships, certifications, preferred roles, and a CV upload.
Input	Manual form entry, ideally pre-filled from Resume Parsing's output.
Output	A saved candidate profile in the shared format.
Approach	Mostly good old coding — a form, basic validation, and a save/load mechanism (JSON). LangChain isn't really needed here. You can use it creatively if you want to.
Tip	Everyone else depends on this. Get a save/load loop working early, even with dummy data, so Roles 4 and 5 aren't blocked waiting on you.

4 · Talent Check

Mission	One button that compares a candidate's profile against a company's expected bar across all 12 RADIX skillsets, and shows the gap.
Input	A saved profile (from Profile Builder) and a selected company.
Output	A readiness score and a per-skillset gap breakdown.
Approach	Mostly comparison and scoring logic — plain code. The company's 12-skillset expectations already exist in RADIX's real data; you'll be given a local snapshot of it for the 3 companies in scope.
Tip	Decide early what “ready” means numerically — a threshold, a weighted score, whatever you choose. That's a design decision, not just an implementation detail, and it changes what the button actually shows.

5 · Skill Matching

Mission	Show how well a candidate matches one specific JD — not a company in general, one actual posting.
Input	A profile or resume, plus one extracted JD skill list from Role 1.
Output	A match score, plus which skills matched and which are missing.
Approach	Comparison logic again, but matching one skill list against another has real fuzziness — the same skill gets described differently in different documents. Good place to use an LLM or simple similarity matching if you have time; exact/fuzzy string matching if you don't.
Tip	A match score with no explanation is much less useful than one that also says what's missing. Prioritize the gap list over the number.

The Shared Data Contract (Starting Point)

This is a proposed shape for the data passing between your five pieces — not a final spec. Your first job, as a team, is to look at this, argue about it, and lock in a version everyone can build against. Change field names if you want. Just don't still be arguing about it at for too long — that's time stolen from Phase 2.

A single extracted or entered skill:

```
{ "skill_name": "...", "category_code":  
"DSA|COD|OOD|APTI|COMM|AI|CLOUD|SQL|SWE|SYSD|NETW|OS|OTHER", "evidence": "short quote or  
reason", "confidence": "high|medium|low" }
```

Output of JD Analytics or Resume Parsing — an extracted skill list:

```
{ "source_type": "jd|resume", "source_file": "...", "company": "...", "role": "...", "skills":  
[ <skill>, ... ] }
```

Output of Profile Builder — a candidate profile:

```
{ "name": "...", "email": "...", "education": "...", "skills": [ <skill>, ... ], "hackathons":  
[...], "internships": [...], "certifications": [...], "preferred_roles": [...], "cv_file":  
"..." }
```

Output of Talent Check:

```
{ "company": "...", "skillset_gap": [ { "category_code": "...", "required_level": 1-10,  
"candidate_level": 1-10, "gap": true|false } ], "readiness_score": 0-100 }
```

Output of Skill Matching:

```
{ "jd_source_file": "...", "match_score": 0-100, "matched_skills": [...], "missing_skills":  
[...] }
```

Your Test Data

A companion document — RADIX Talent Match Hackathon: Job Descriptions & Sample Candidate Profiles — gives you everything you need to build and test against:

- 3 companies, 2 roles each: Google (Software Engineer, Data Scientist), Microsoft (Software Engineer, Data Analyst), and Oracle Financial Services Software (Associate Software Engineer, Application Support Analyst) — 6 real JDs.
- 4 sample candidate resumes with deliberately different skill profiles — a systems-leaning generalist, a data-leaning analyst, an entry-level support-oriented candidate, and a strong all-rounder — so Talent Check and Skill Matching results should visibly differ from person to person.

Talent Check needs one more thing: each company's expected level across all 12 skillsets. Since we're using local files today rather than live Supabase, you'll be handed a local JSON snapshot of that data for these 3 companies — ask your facilitator for `talent_check_company_skillsets.json` if you don't see it in your starter folder.

The Day: Four Phases

Time	Phase	What Happens
1 Hour	1. Planning on Integration	Whole team together. Read this document and the JD/resume pack. Agree on the shared data contract (adjust the starting point above as needed) and on names/locations. Everyone should leave this hour knowing exactly what they're building and exactly what shape their output needs to be.
2 Hours	2. Development	Heads down, building your own piece solo. Test against the sample JDs/resumes as you go — don't wait until integration to discover your JSON doesn't parse.
1 Hour	Break	Lunch.
1.5 Hours	3. Integration	Bring all 5 pieces together into one working flow: pick a JD → parse a resume → build/update a profile → run Talent Check → run Skill Matching. Budget the first 15 minutes for “why won't this even run together” — it never goes perfectly on the first try.
TBD	4. Presentation	Timing and format announced separately — see Presentation below.

Tech Stack & Guidance

- Python throughout — same stack as the rest of RADIX.
- Use LangChain or an LLM agent wherever it genuinely helps — JD Analytics, Resume Parsing, and the fuzzy-matching part of Skill Matching are strong candidates. Where plain code is simpler and more reliable — Profile Builder's form, Talent Check's scoring logic — use plain code. An agent that's harder to debug than the problem it solves isn't a win.
- Frontend: React, matching the RADIX portal's stack.
- Data storage: Supabase — JSON for most things.

What “Done” Looks Like

- All 5 pieces run on their own against at least 2 of the sample JDs/resumes
- The 5 pieces are wired together into one flow, end to end, for at least one company / JD / candidate combination
- Talent Check and Skill Matching produce visibly different results for different sample candidates — the scoring logic isn't a constant

Presentation

Exact timing and format will be confirmed. Plan for a short live walkthrough of your integrated tool — not just a slide deck.